COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/16/26	**DATE SUBMITTED:**	10/16/26
NAME:	SUBALISID, JAPPETH ROYCE B. BAUTISTA, MARK JONEIL S.	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

Hands-on Activity 13.1	**Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

ILO A & ILO B:

Part 1: Simple One-Threaded Example

Since all threads must be given a function to complete at their creation, we first must declare a function for it to be given. We'll name this function print and will design it to take int and string arguments when called. When executed, this code will simply report the data values passed in.

```
void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}
```

In the next section, we'll initialize a thread and have it execute the above function. To do this, we'll have the main function, the default executor present in all C++ applications, initialize the thread for the print function.

After that, we use another handy multithreading command, join(), pausing the main function's thread until the specified thread, in this case t1, has finished its task. Without join() here, the main thread would finish its task before t1 would complete print, resulting in an error.

```
int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

Screenshot

```
Printing integer: 10
Printing string: T.I.P.
```

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis	The program initializes a single worker thread, designated as t1 which executes the print function with specific parameters while the main thread waits. Since there is only one thread and it's also the only function called in the main driver function, the output looks like a simple sequential execution, which is to be expected.The program initializes a single worker thread, designated as t1 which executes the print function with specific parameters while the main thread waits.

Table 13-1. Simple One-Threaded Example

Part 2: Multi-Threaded Example

While the outcome of the single thread example above could easily be replicated without using multithreaded code, we can truly see concurrency's benefits when we attempt to complete print multiple times with different sets of data. Without multithreading, this would be done by simply having the main thread repeat print one at a time until completion.

To do this with concurrency in mind, we instead use a for loop to initialize multiple threads, pass them the print function and arguments, which they then complete concurrently. This multithreading option would be faster using only the main thread as more of the total CPU is being used.

Runtime difference between multithreading and non-multithreading solutions increases as more print executions are needed.

Let's see what a many-thread version of the above code would look like:

```
void print(int n, const std::string &str) {
std::string msg = std::to_string(n) + " : " + str;
std::cout << msg << std::endl;
}
int main() {
std::vector<std::string> s = {
"T.I.P.",
"Competent",
"Computer",
"Engineers"
};
std::vector<std::thread> threads;
for (int i = 0; i < s.size(); i++) {
threads.push_back(std::thread(print, i, s[i]));
}
for (auto &th : threads) {
th.join();
}
return 0;
}
```

Screenshot	`1 : Competent2 : Computer0 : T.I.P.3 : Engineers`

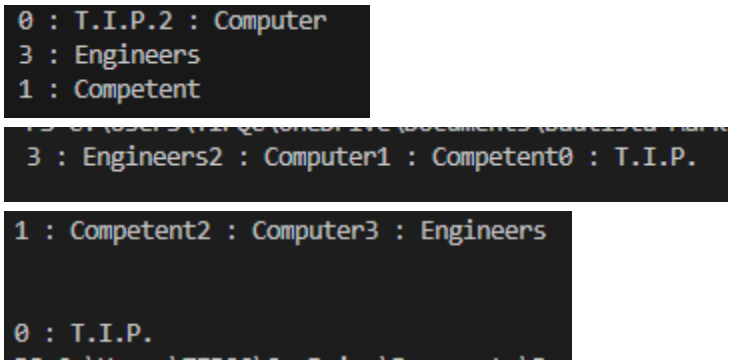
	
Analysis	The program utilizes a loop to spawn a vector of multiple threads, handing over execution and controls to the operating system's scheduler to assign them to available CPU cores. Since the threads are working simultaneously, the function runs in random order printing a different sequence every time the main driver is run. Because these threads execute concurrently and independently without a strict synchronization lock on standard output, the terminal output lines interleave unpredictably, demonstrating the asynchronous nature and race conditions inherent in multithreaded environments.

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

COMPUTER ENGINEERING

QUEZON CITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

1. Write a definition of multithreading and its advantages/disadvantages.

Bautista	Multithreading allows a single process to execute multiple threads concurrently while sharing the same memory and system resources. Its key advantages are enhanced responsiveness, better multi core performance and lower resource overhead compared to separate processes. However it increases code complexity and introduces risks like race conditions, deadlocks and difficult debugging.
Subalisid	Multithreading is the process where a program runs multiple functions/threads simultaneously. This has the advantage of maximizing the processing speed of code. Since some tasks require wait time before being executed, multithreading allows the program to switch to a different thread while waiting for the others to finish. However, as seen in the procedure activity, this has the drawback of being unpredictable since the functions are being executed simultaneously leading to different outputs between executions.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

Bautista	Multithreading can be used in many cases, in GUI applications, background threads handle heavy computations or network calls without freezing the user interface. Web servers use multithreading to manage thousands of simultaneous client requests efficiently. In video games, separate threads run physics, AI and graphics simultaneously to maintain smooth performance.
Subalisid	Multithreading is a great practice when it comes to having an interactive UI where there can be multiple things happening at once. Video games for example can handle the task of loading the map visuals while also taking in the player's controls. This allows the user to move freely while the background environment is still loading or not in its best quality. Music streaming platforms also can utilize this as they have the functionality of browsing songs while a song is still playing, with others even allowing videos, comments, download functionality without stopping the song from playing continuously. Lastly, this google docs is also one of the best examples. It allows multiple users to input simultaneously, while also checking for spelling/grammatical errors and auto saving to the cloud periodically.

3. Differentiate between parallelism and concurrency.

Bautista	Concurrency is about managing multiple tasks at once by interleaving their execution through rapid switching, even on a single processor. Parallelism is physically executing multiple tasks at the exact same instant using multi core hardware. Simply put concurrency handles many things at once, while parallelism does many things at once.
Subalisid	Parallelism is when two tasks are being handled at the same time but uses different parts of the processing unit, therefore not continuously switching between tasks and allowing for faster general execution. Concurrency is when multiple tasks are handled by a single processing unit. This utilizes task switching to optimize runtime, allowing other tasks to run while the others are waiting for an input/output.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

- Create a global variable of type integer.
- Create an add function that will take an integer parameter and add that value to the global variable.
- Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
- Display the value of global variable at different combinations of using the join() per thread. Such that:
  - Display
  - T1.join()
  - Display
  - T2.join()
  - Display
  - T3.join()
- Provide an analysis based on the outputs of the multi-threading exercise.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

```
#include <iostream>
#include <thread>

int global;

void add(int n) {
   global += n;
}

int main() {
   std::cout << "Starting global value: " << global << std::endl;
   std::thread t1(add, 3);
   std::thread t2(add, 6);
   std::thread t3(add, 9);

   std::cout << "Current global value: " << global << std::endl;
   t1.join();
   std::cout << "Current global value: " << global << std::endl;
   t2.join();
   std::cout << "Current global value: " << global << std::endl;
   t3.join();
   std::cout << "Current global value: " << global << std::endl;
```

```
  return 0;
}
```

```
Starting global value: 0
Current global value: 0
Current global value: 18
Current global value: 18
Current global value: 18
```

Bautista	The program declares a global integer and a function that increments it. It prints the starting value of 0 before spawning three threads to add 3, 6 and 9 concurrently. Because the main thread continues immediately, early print statements may show zero before it updates complete. As t1, t2, t3 are sequentially joined, the main thread waits for them to finish, safely accumulating the total sum. The final output reflects the complete sum of 18 at the start, but when we change the operation to multiplication instead of adding them the final output reflects the results 162, 27 and 54 which is different from the first final output.
Subalisid	After the first starting value print and the first global value print, we get 0 because the threads weren't done with their execution, but immediately after thread 1 finishes, the result already shows that all three threads had already finished and shows the total value as 18. This can imply the execution speed of the threads in terms of a simple addition function. However, after testing, changing the global to have a default of 1 and changing the operation to multiply instead of add, we get the results 162, 27, and 54 which are all different.

3. CONCLUSION

This activity helped us understand the workings of multithreading using experimental codes. We saw how each output changes as a result of simultaneous execution among threads with the procedure activity. This was further affirmed by the supplementary activity experiment where addition worked completely fine but multiplication caused some randomness in the output. Overall, this activity might be useful in the future when we are already utilizing multithreading in most of our lab activities. This shows how a thread can override a previous update of another thread, resulting in unpredictable outputs. This activity helped us understand the workings of multithreading using experimental codes. We saw how each output changes as a result of simultaneous execution among threads

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.